

Feedback-Augmented RT-DETR

for Small Object Detection

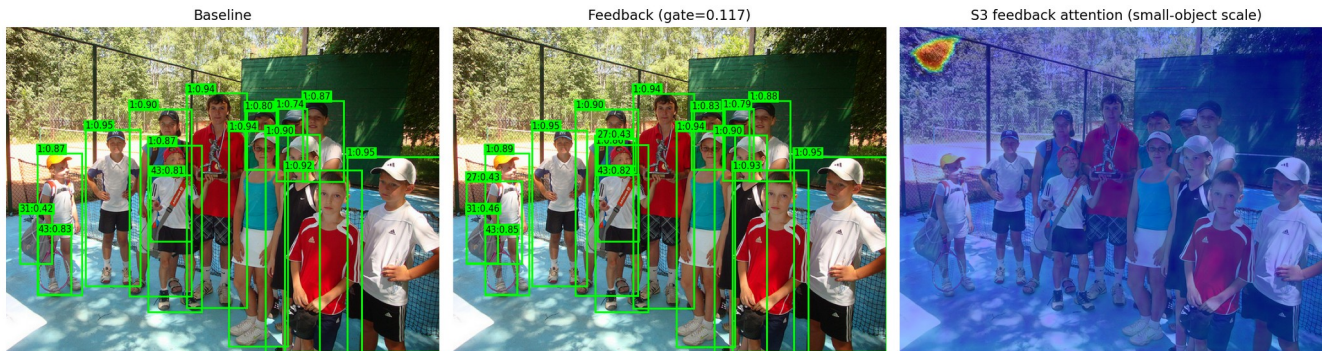
Hatem Saadallah · Nour Jennane

Final Project · April 2026

Problem Formulation

What is small-object detection, and why does RT-DETR struggle?

Small objects are harder to detect



AP_L (large)

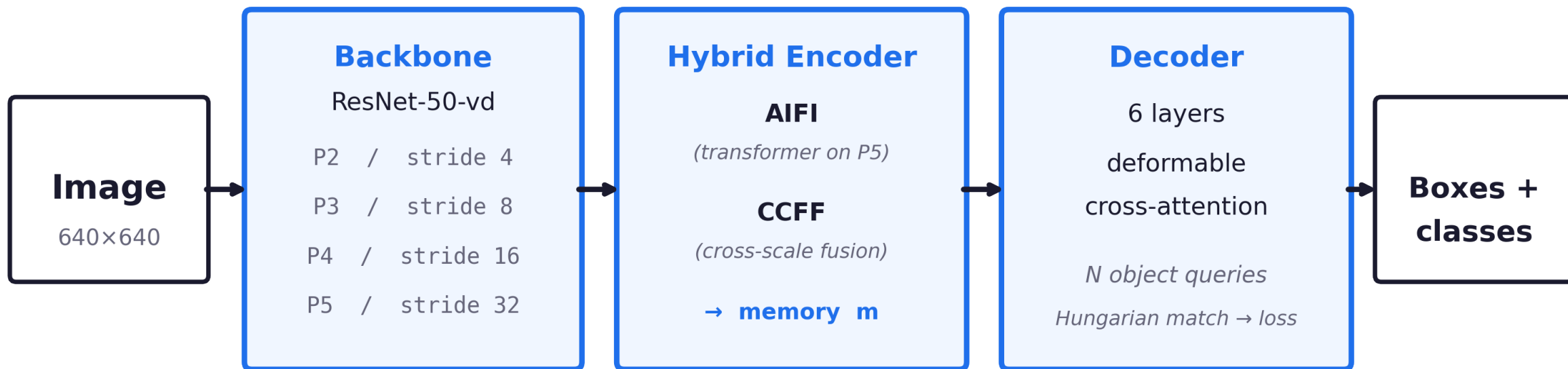
67.7

AP_S (small)

34.7

33-point gap. Why?

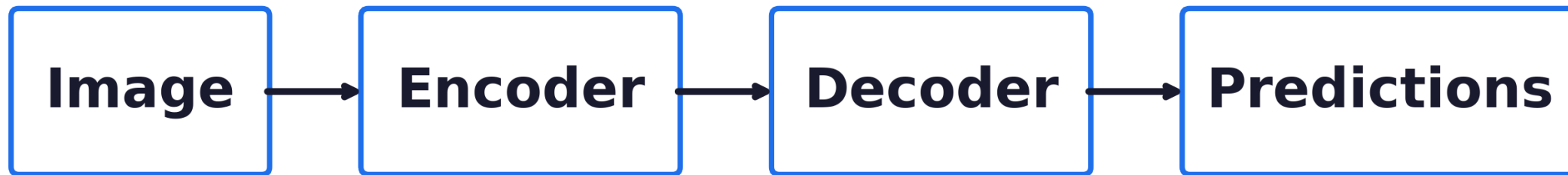
RT-DETR: backbone → encoder → decoder



End-to-end, NMS-free. Six decoder layers all attend over the same encoder memory.

Backbone extracts a feature pyramid (P2–P5); the hybrid encoder fuses scales; the decoder cross-attends.

The reason is structural



one-way information flow

Encoder memory is computed once. The decoder cannot revise it.

Refinement of the features small objects live in is a one-way street.

Data Sourcing Strategy

What dataset we used and why it is appropriate.

COCO 2017 — the standard small-object benchmark

Dataset

Microsoft COCO 2017

Splits

train2017: 118k images, 860k boxes

val2017: 5k images (held out)

Pre-trained weights

rtdetr_r50vd_6x_coco.pth (public, finetuned 13 ep)

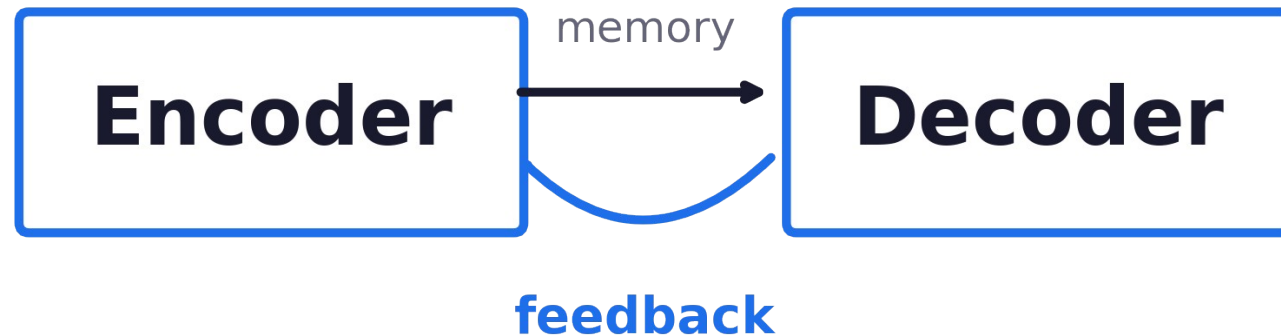
Why COCO?

- Standard benchmark — directly comparable to RT-DETR, DETR, YOLO.
- Has explicit small / medium / large size buckets (S / M / L).
- ~41% of annotated boxes are small (area < 32² px).
- Same pre-training corpus as our baseline → fair comparison.

Proposed Solution

The feedback module, the math behind it, and the two fixes that make it work.

Our idea: let the model refine its own features



Feed early decoder predictions back into the encoder memory.

Attention, in one line

$$\text{Attn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\right) \mathbf{V}$$

Q

what we look for
(query)

K

where we look
(addresses)

V

what we extract
(contents)

Attention lets the model focus on important parts of the image.

Our feedback rule

$$\mathbf{m}' = \mathbf{m} + g \cdot \text{Attn}(\mathbf{m}, \mathbf{t})$$

m

encoder memory
(image features)

g

gate
(how much feedback)

t

decoder output
(early predictions)

We update the encoder using the decoder's own predictions.

First attempt (v1) — and why it failed

Plain sigmoid gate, applied to all four pyramid levels.

$$\Delta AP_s = 0.00$$

the mechanism contributed nothing

Why?

α drifts to $-\infty \rightarrow \text{gate} \approx 0$.

The feedback signal is multiplied by zero before reaching memory.

First attempt (v1) — and why it failed

Plain sigmoid gate, applied to all four pyramid levels.

$$\Delta AP_s = 0.00$$

the mechanism contributed nothing

Why?

α drifts to $-\infty \rightarrow \text{gate} \approx 0$.

The feedback signal is multiplied by zero before reaching memory.

v2: two fixes — zero new parameters

1. Gate floor

$$g_{\text{eff}} = 0.1 + 0.9 \sigma(\alpha)$$

feedback can never be silenced

2. P2 / P3 only

refine where small objects live

skip P4, P5 — focus the gradient

Reparameterize the gate. Refine only the levels small objects live in.

Performance Evaluation Approach

Metrics, baselines, and the same-checkpoint ablation that isolates causality.

How we evaluate the contribution

Metrics (COCO standard)

AP, AP_S, AP_M, AP_L — IoU 0.5..0.95

AP_S is the headline metric (area < 32² px)

Others verify we don't degrade easier categories.

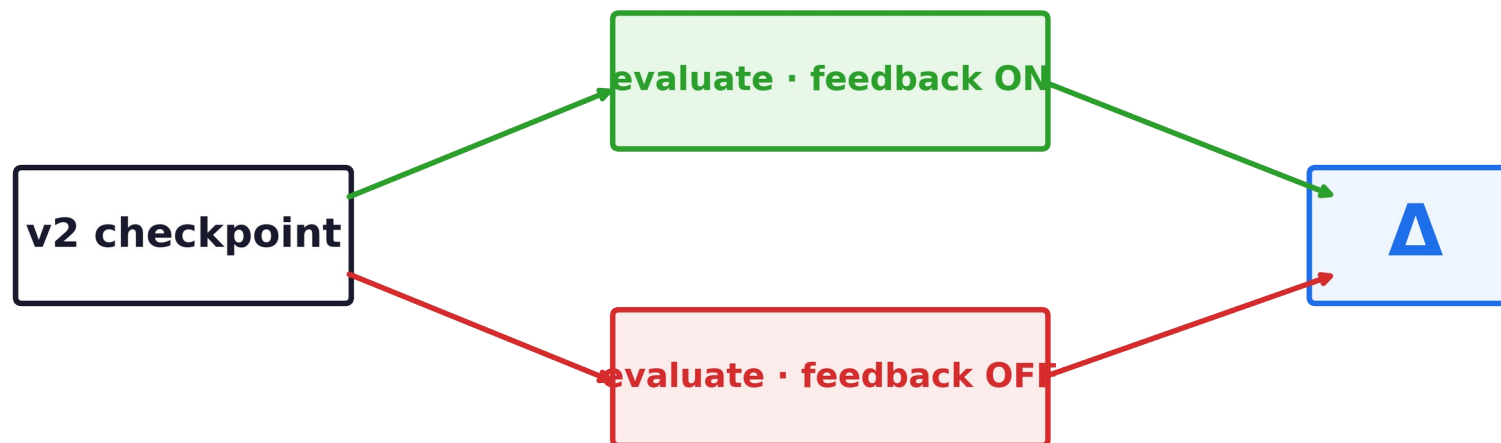
Baselines

RT-DETR-R50 (published)

v1 feedback (before the fix)

v2 feedback (with the fix)

Same-checkpoint ablation — isolates causal contribution

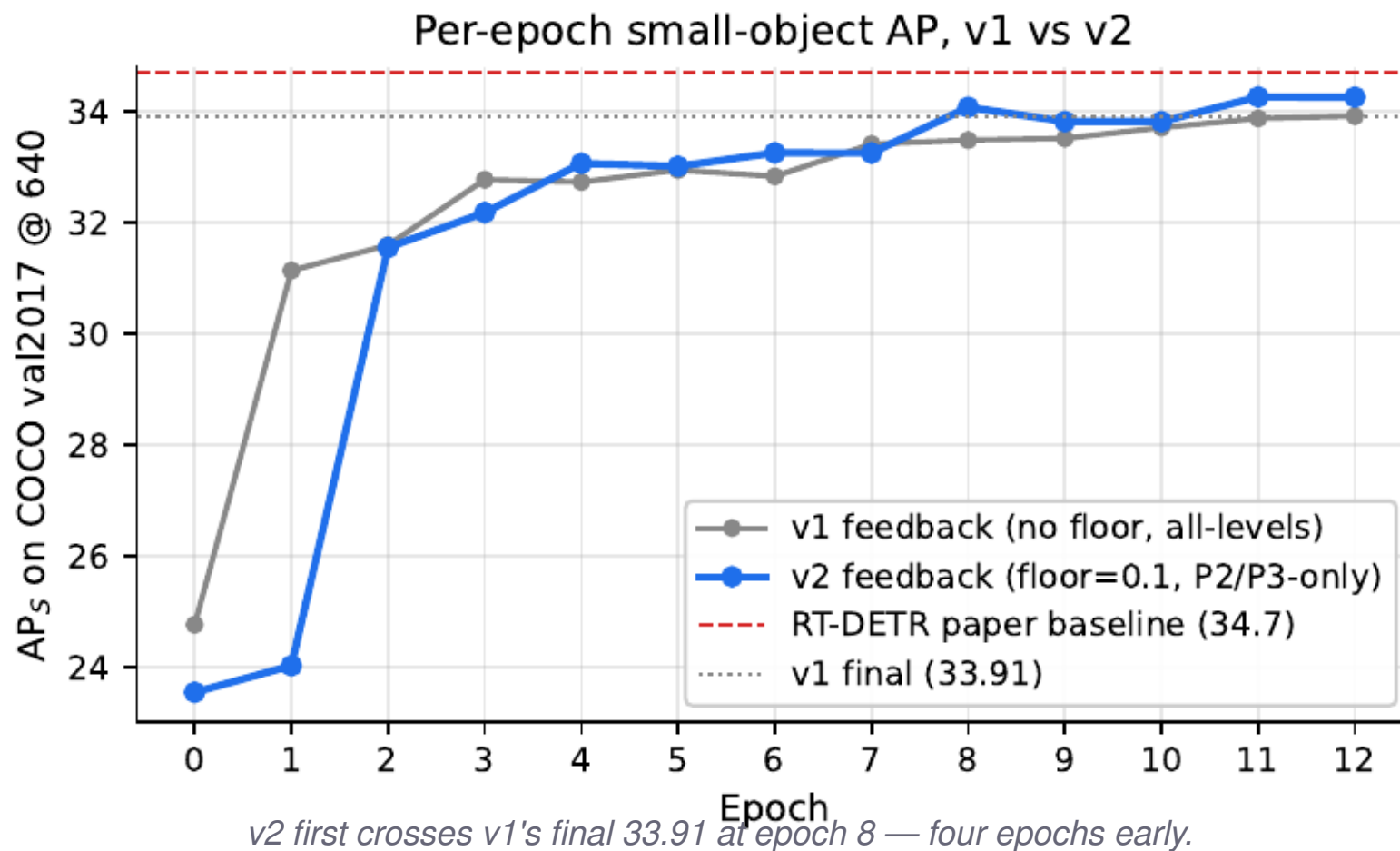


Toggle feedback ON / OFF at inference. Identical weights, identical inputs.

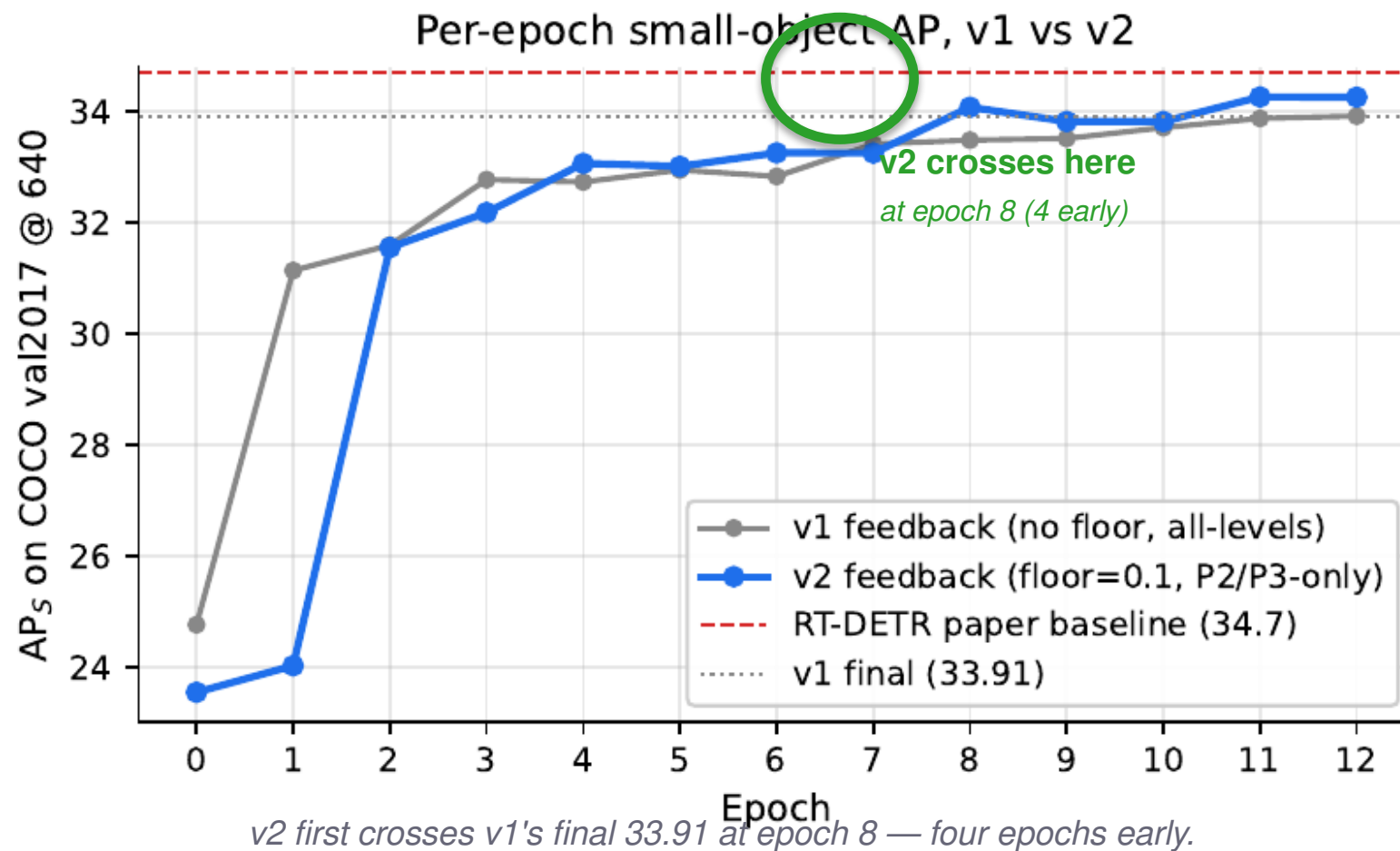
Final Results

Training trajectory, the causal effect, robustness, trade-offs, and what's next.

Training: v2 climbs faster, ends higher



Training: v2 climbs faster, ends higher



Final performance (with feedback ON)

v1

33.91

AP_s

v2

34.30

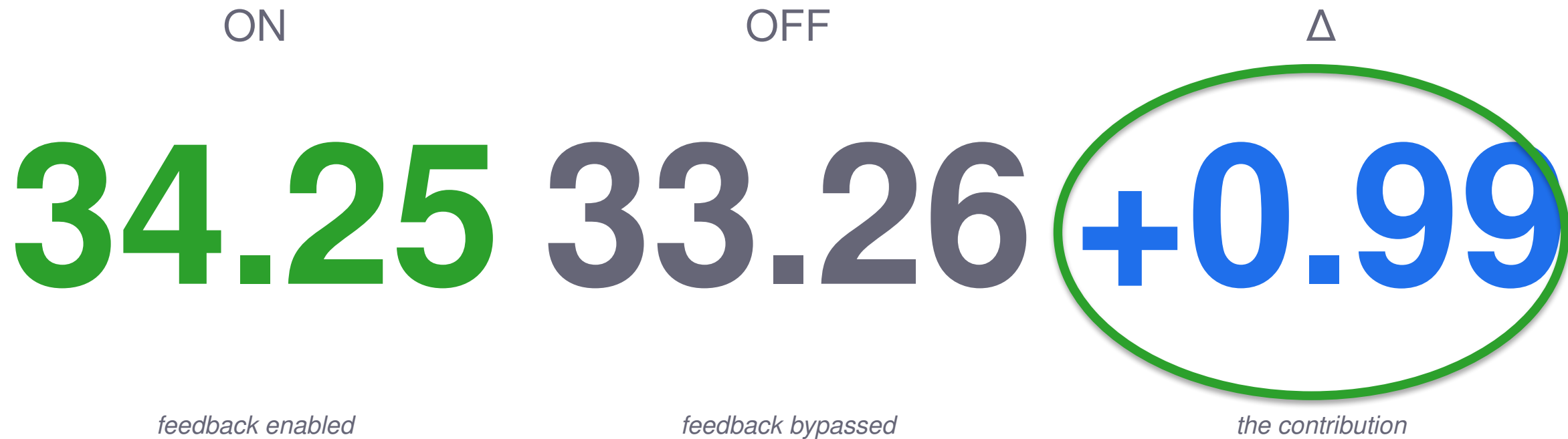
$AP_s +0.4$

Causal effect of feedback

ON	OFF	Δ
34.25	33.26	+0.99
<i>feedback enabled</i>	<i>feedback bypassed</i>	<i>the contribution</i>

Feedback has a real causal impact on small-object detection.

Causal effect of feedback



Feedback has a real causal impact on small-object detection.

Higher resolution → bigger gain

640 × 640

800 × 800

34.25 → 37.01

+2.76 from more pixels.

Feedback helps. Spatial sampling helps too.

Strengths and limitations

+ Strengths

- Causal contribution: +0.99 on small-object AP, same checkpoint.
- Zero new parameters relative to v1.
- Consistent improvement on every metric (AP, S, M, L).
- Robust at higher resolution (37.01 small-object AP at 800×800).

– Limitations

- Single seed — multi-seed variance unmeasured.
- 13-epoch finetune (vs 72-epoch from scratch).
- Floor and mask not isolated (a 2×2 ablation is missing).
- $\approx 2\times$ slower than baseline (53 $\rightarrow$ 26 FPS) — cost is the P2 level.

Strong on accuracy; pays in inference speed. The next slide proposes a way to recover speed.

Future work: recovering inference speed

The 2× slowdown comes from the P2 level, not the feedback module.

1

Train with P2, infer without

fastest path

How

Use P2 during training to improve small-object learning, then remove it at inference.

Goal

Keep most of the +0.99 small-object AP gain without the P2 computational cost.

2

Lighter P2 path

compromise

How

Reduce the cost of P2 — fewer channels, or a simpler projection layer.

Goal

Preserve small-object improvements while reducing the high-resolution P2 slowdown.

3

Knowledge distillation

principled

How

Transfer v2's small-object improvements into a faster baseline RT-DETR student.

Goal

Recover baseline inference speed while retaining the small-object AP gains learned from P2 feedback.

Common goal: keep the +0.99 small-object AP gain while approaching the 53 FPS baseline.

Conclusion

1 Feedback improves small-object detection

+0.99 small-object AP, causal contribution at inference.

2 Only works with proper design

Floor the gate; restrict to small-object levels.

3 Modest but real

Consistent across metrics; reproducible from the same checkpoint.

Thank you.

Questions?



Hatem Saadallah · Nour Jennane

github.com/HatemSaadallah/feedback-augmented-rtdetr